{vmt DEV

/ bootcamp
1º edición /

Angular

Bootcamp Tercera Edición

Componentes y Templates

Módulo 2

{vmt DEV
/ bootcamp
1ª edición /

Desglose del módulo

1. Generalidades de TypeScript
2. Creación de **componentes**
3. Decorador **@Component**
4. Interpolación de datos
5. Property binding
6. Event binding
7. Two-way binding con ngModel
8. Directivas **estructurales** (ngIf, ngFor)
9. Directivas de **atributos** (ngClass, ngStyle)

1. Generalidades de TypeScript

¿Qué es TypeScript?

- TypeScript es JavaScript con tipado estático. Cuando escribes código en Angular, estás escribiendo TypeScript que luego se compila a JavaScript normal para el navegador.

- La ventaja principal es que el editor (y el compilador) te avisan de errores *antes* de ejecutar el código.

Variables: Generalidades

- **const** nombreCurso: string = “Curso de Angular";
- **let** esDificil: boolean = false;
- **let** totalClases: number = 15;

Variables – String

- **let** nombre: string = "Ana";
- **let** saludo: string = `Hola, ${nombre}`;
- let precio: number = 15.5;

- **let** msg = `Total: ${precio * 2} dólares`;
- **let** msg = `Aprobado: ${nota >= 7 ? "Sí" : "No"}`;
  let msg = `Materias: ${materias.length}`;

Variables – Number

- **let** edad: number = 21;
- **let** promedio: number = 8.75;

Variables – Boolean

- **let** estaLogueado: boolean = false;
- **let** tienePermiso: boolean = true;

NULL y ANY

- **let** usuario: string | null = null;
- **let** resultado: any;

Estructura de datos - Arreglos

- **let** materias: string[] = ["Cálculo", "Física", "Programación"];
- **let** notas: number[] = [8.5, 9.0, 7.3];
- **let** estados: boolean[] = [true, false, true];

- materias.push("Base de Datos");
  materias.pop();
- materias.length;
- materias[0];

Interfaces y Clases

{ "nombre": "Ana", "edad": 21, "carrera": "Sistemas" }

```
interface Estudiante {
 nombre: string;
 edad: number;
 carrera: string;
}
```

```
let estudiante: Estudiante = respuestaApi;
console.log(estudiante.nombre);
```

Describe propiedades

Interfaces y Clases

```
{ "nombre": "Ana", "edad": 21, "carrera":
"Sistemas" }
```

```
class Estudiante {
 constructor(
  public nombre: string,
  public edad: number,
  public carrera: string
) {}}
```

```
let estudiante =
new Estudiante(
respuestaApi.nombre,
respuestaApi.edad,
respuestaApi.carrera);
```

```
console.log(estudiante.nombre);
```

Describe métodos y lógica

```
class Estudiante {
  constructor(
    public nombre: string,
    public apellido: string,
    public notas: number[]
  ) {}

  nombreCompleto(): string {
    return `${this.nombre} ${this.apellido}`;
  }

  promedio(): number {
    const suma = this.notas.reduce((acc, n) => acc + n, 0);
    return suma / this.notas.length;
  }

  estaAprobado(): boolean {
    return this.promedio() >= 7;
  }
}
```

Estructura de datos – Arreglo de objetos

```
interface Materia {
  nombre: string;
  creditos: number;
  aprobada: boolean;
}

materias: Materia[] = [
  { nombre: "Cálculo",  creditos: 4, aprobada: true  },
  { nombre: "Física",  creditos: 4, aprobada: false },
  { nombre: "Programación", creditos: 3, aprobada:
true  },
];
```

2. Creación de componentes

Un componente es la unidad básica de Angular. Combina lógica (TypeScript), vista (HTML) y estilos (CSS).

```
PS C:\Users\USER\Desktop\mi-proyecto-angular> ng generate component saludo-component
CREATE src/app/saludo-component/saludo-component.spec.ts (615 bytes)
CREATE src/app/saludo-component/saludo-component.ts (237 bytes)
CREATE src/app/saludo-component/saludo-component.scss (0 bytes)
CREATE src/app/saludo-component/saludo-component.html (32 bytes)
PS C:\Users\USER\Desktop\mi-proyecto-angular>
```

El CLI crea y registra el componente automáticamente en el módulo más cercano.

3. Decorador @Component

El decorador le dice a Angular cómo funciona el componente: su selector HTML, su template y sus estilos.

```
1   import { Component } from '@angular/core';
2
3   @Component({
4     selector: 'app-saludo-component',
5     imports: [],
6     templateUrl: './saludo-component.html',
7     styleUrl: './saludo-component.scss',
8   })
9   export class SaludoComponent {
10
11  }
```

src > app > app.ts > ...

```
1   import { Component } from '@angular/core';
2   import { RouterOutlet } from '@angular/router';
3   import { SaludoComponent } from './saludo-component/saludo-component';
4
5   @Component({
6     selector: 'app-root',
7     imports: [RouterOutlet, SaludoComponent],
8     templateUrl: './app.html',
9     styleUrl: './app.scss'
10  })
11  export class App {
12    protected title = 'mi-proyecto-angular';
13  }
```

src > app > app.html > ...

Go to component

```
1  <h1>¡Hola! Esta es mi página de inicio</h1>
2  <app-saludo-component />
3  <router-outlet />
```

4. Interpolación de datos

La **clase** tiene datos, el **template** los muestra.

```
import { Component } from "@angular/core";

@Component({
  selector: 'app-app-component',
  standalone: true,
  templateUrl: './app-component.html',
})
export class AppComponent {
  titulo: string = "Curso de Angular";
  estudiante: string = "Luis Gómez";
  activo: boolean = true;
}
```

4. Intepolación de datos

```
Go to component
<h1>Hola, mi clase se llama: {{ titulo }}</h1>
<p>Yo me llamo {{ estudiante }}, un gusto.</p>
```

Interpolación {{ }}
Muestra el valor de una propiedad dentro del HTML:

- <h1>{{ titulo }}</h1>
- Bienvenido, {{ estudiante }}
- Año: {{ 2025 }}
- Suma: {{ 10 + 5 }}

5. Property binding

Conecta una propiedad de la clase con un **atributo HTML**.
Lo que cambia en la clase, cambia en el HTML automáticamente:

```
export class AppComponent {

  imagenUrl: string = "https://picsum.photos/200";
  botonDesactivado: boolean = true;
  claseCSS: string = "texto-rojo";

}
```

5. Property binding

```
Go to component
<img src="https://picsum.photos/200">

<img [src]="imagenUrl">
<button [disabled]="botonDesactivado">Enviar</button>
<p [class]="claseCSS">Este texto cambia de clase</p>
```

La diferencia entre interpolación y property binding:

- La **interpolación** se usa para mostrar texto.
- El **property binding** se usa para atributos HMTL.
- Se usan corchetes [] para definir el atributo.
- `style`, `src`, `class`, etc.

6. Event binding

Conecta un evento del HTML con un método de la clase

```
export class AppComponent {
  contador: number = 0;

  incrementar(): void {
    this.contador++;
  }

  mostrarAlerta(): void {
    alert("Hola!");
  }
}
```

6. Event binding

Go to component

```
<button (click)="incrementar()">+1</button>
<button (click)="mostrarAlerta()">Saludar</button>
<p>{{ contador }}</p>
```

Cada vez que el usuario hace clic, Angular ejecuta el método. El template se actualiza solo.
Se usan paréntesis () para definir el evento.

7. Two-way binding

Es la combinación de property binding y event binding en uno solo. Angular los fusiona con [(ngModel)].

```
<input [value]="nombre">
```

Esto muestra el valor de nombre en el input. Pero si el usuario escribe algo, la variable nombre en la clase no se entera. El dato solo va en una dirección.

```
<input [value]="nombre" (input)="nombre =
$event.target.value">
```

7. Two-way binding

```
<input [(ngModel)]="nombre">
```

- ngModel es una directiva que sincroniza input y variable.
- FormsModule es el módulo que contiene a ngModel
- Solo se utiliza en elementos que reciben un input por parte del usuario.

Ejercicio 1

1. Clara quiere mostrar el nombre de su aplicación en un <h1>. El nombre está guardado en una variable de la clase. ¿Qué debe usar?

- [innerText]
- {{ }}
- [(ngModel)]
- (click)

Ejercicio 1

1. Clara quiere mostrar el nombre de su aplicación en un <h1>. El nombre está guardado en una variable de la clase. ¿Qué debe usar?

- [innerText]
- {{ }}
- [(ngModel)]
- (click)

Respuesta

b. Se usa la interpolación para mostrar el valor de una variable dentro del HTML como texto. El <h1> no es un atributo, es contenido de texto, y para eso se usa {{ }}.

2. Pedro tiene una variable urlFoto: string en su clase y quiere que una etiqueta <img> cargue esa imagen dinámicamente. ¿Qué debe usar?

- {{ urlFoto }}
- (src)="urlFoto“
- [src]="urlFoto“
- [(src)]="urlFoto"

2. Pedro tiene una variable urlFoto: string en su clase y quiere que una etiqueta <img> cargue esa imagen dinámicamente. ¿Qué debe usar?

- {{ urlFoto }}
- (src)="urlFoto“
- [src]="urlFoto“
- [(src)]="urlFoto"

Respuesta

c. [src]="urlFoto"src es un atributo HTML. Para conectar un atributo HTML con una variable de la clase se usa property binding con corchetes [ ].

3. Luis tiene un botón "Me gusta" y quiere que cada vez que el usuario haga clic se ejecute el método agregarLike() de su clase. ¿Qué debe usar?

- [click]="agregarLike()“
- (click)="agregarLike()“
- {{ agregarLike() }}
- [(click)]="agregarLike()"

3. Luis tiene un botón "Me gusta" y quiere que cada vez que el usuario haga clic se ejecute el método agregarLike() de su clase. ¿Qué debe usar?

- [click]="agregarLike()“
- (click)="agregarLike()“
- {{ agregarLike() }}
- [(click)]="agregarLike()"

Respuesta

b. (click)="agregarLike()"click es un evento del navegador. Para escuchar eventos se usan paréntesis ( ). Cuando el usuario hace clic, Angular ejecuta el método.

4. Carlos tiene un componente con un contador. Cada vez que el usuario hace clic en "Incrementar", el número en pantalla debe cambiar. Tiene el método incrementar() y la variable contador. ¿Qué combinación necesita en el template?

- {{ contador }} y (click)="incrementar()“
- [contador] y [click]="incrementar()“
- [(contador)] y (click)="incrementar()“
- {{ incrementar() }} y [click]="contador"

4. Carlos tiene un componente con un contador. Cada vez que el usuario hace clic en "Incrementar", el número en pantalla debe cambiar. Tiene el método incrementar() y la variable contador. ¿Qué combinación necesita en el template?

- {{ contador }} y (click)="incrementar()“
- [contador] y [click]="incrementar()“
- [(contador)] y (click)="incrementar()“
- {{ incrementar() }} y [click]="contador"

Respuesta

a. {{ contador }} y (click)="incrementar()"Son dos cosas independientes haciendo cada una su trabajo. {{ contador }} muestra el valor actual. (click)="incrementar()" ejecuta el método que actualiza la variable. Cuando la variable cambia, Angular actualiza el {{ contador }} automáticamente.

8. Directivas en Angular

- Una directiva es una instrucción que le das a Angular sobre cómo debe comportarse un elemento HTML.
- No es una etiqueta HTML nueva — es algo que le pegas encima a una etiqueta existente para cambiar su comportamiento.

Hay tres tipos:

- Componentes
- Estructurales
- De atributo

8. Directivas Estructurales

***ngIf - @if**

```
<p *ngIf="estaLogueado">Bienvenido</p>

<p *ngIf="estaLogueado; else noLogueado">
Bienvenido</p>
<ng-template #noLogueado>
   <p>Debes iniciar sesión</p>
</ng-template>
```

8. Directivas Estructurales

A partir de Angular 17+

```
@if (estaLogueado) {
  <p>Bienvenido</p>
}
@else {
  <p>Debes iniciar sesión</p>
}
```

8. Directivas Estructurales

***ngFor - @for**

```
<li *ngFor="let materia of materias">
   {{ materia.nombre }}
</li>

@for (materia of materias; track materia.nombre) {
  <li>{{ materia.nombre }}</li>
}

@for (materia of materias; track $index) {
   <li>{{ materia.nombre }}</li>
}
```

8. Directivas Estructurales

***ngFor - @for**

```
@for (materia of materias; track materia.nombre) {
  <li>{{ materia.nombre }}</li>
} @empty {
  <li>No hay materias registradas</li>
}
```

8. Directivas de atributo

A diferencia de las estructurales que agregan o eliminan elementos, las directivas de atributo **modifican la apariencia o el comportamiento** de un elemento que ya existe.

[class]

[class.nombreDeLaClase]="condición“
Es property binding aplicado a una clase CSS.

- Si la condición es true, la clase se aplica.
- Si la condición es false, la clase se quita.

8. Directivas de atributo

CSS

```
.aprobado { color: green; }
```

HTML

```
<p [class.aprobado]="materia.aprobada">{{
materia.nombre }}</p>
```

8. Directivas de atributo

[style]

[style.propiedadCSS]="valor“
A diferencia de [class.], aquí no se definee la clase en el CSS, sino que el estilo se aplica directamente sobre el elemento. El valor es dinámico y calculado en tiempo real.

- [style.color]="colorVariable“
- [style.color]="nota >= 7 ? 'green' : 'red’”
- [style.fontSize.px]="16"

Usa **[class.]** cuando los estilos ya están definidos en el CSS y solo necesitas decidir cuándo aplicarlos

```
<p [class.aprobado]="materia.aprobada"
[class.reprobado]="!materia.aprobada">
{{ materia.nombre }}
</p>
```

Usa **[style.]** cuando el valor es calculado y no puedes definirlo en el CSS de antemano

```
<div [style.width.%]="progreso"></div>
```

Usa **[class.]** cuando los estilos ya están definidos en el CSS y solo necesitas decidir cuándo aplicarlos

```
<p [class.aprobado]="materia.aprobada"
[class.reprobado]="!materia.aprobada">
{{ materia.nombre }}
</p>
```

Usa **[style.]** cuando el valor es calculado y no puedes definirlo en el CSS de antemano

```
<div [style.width.%]="progreso"></div>
```

Ejercicio 2

Elabore un aplicativo que simule el portal de un estudiante universitario.

{vmt DEV

/ bootcamp
1° edición /

www.vmt.dev

@vmtdev